

Projeto 2 – Desenho de Algoritmos

GRUPO 2 – TURMA 2

DIOGO VIEIRA – UP202208723

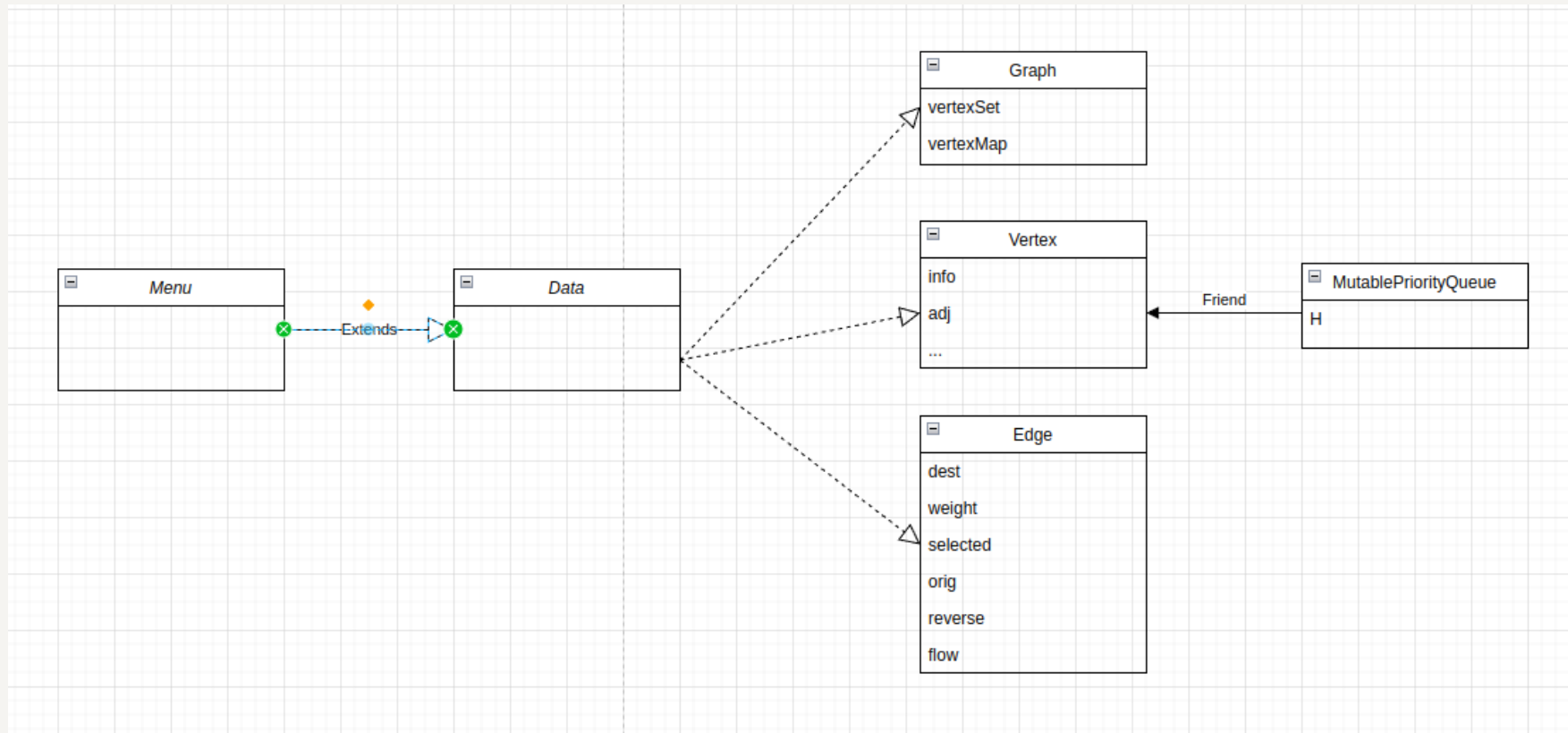
ANTERO CABRAL – UP202204971

TIAGO PINTO - UP202206280

Estrutura Utilizada

- Para conseguirmos guardar diferentes tipos de vértices, utilizamos ID's únicos para cada vértice, que são armazenados na estrutura do grafo "network_". No caso dos pontos turísticos, ou seja, o ficheiro "tourism.csv", utilizamos um mapa "tourismLabels" para armazenar rótulos adicionais associados aos vértices.
- Os vértices são adicionados ao grafo utilizando o método addVertex, que armazena o ID, longitude, latitude, e um estado booleano indicando se o vértice é válido. As arestas, que representam conexões entre os vértices, são adicionadas com o método addEdge, que utiliza os IDs dos vértices de origem e destino e o peso da aresta.

Diagrama de Classes



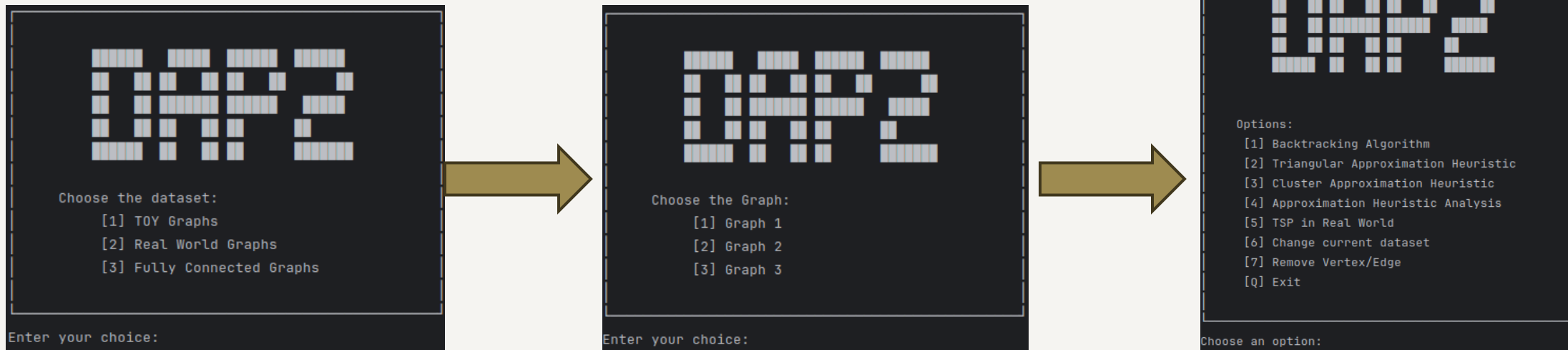
Menu Inicial

- No menu principal podemos fazer a escolha dos ficheiros, que escolhemos para popular o nosso programa. De facto, podemos escolher entre os grafos "Toy", "Fully Connected" e os "Real World".

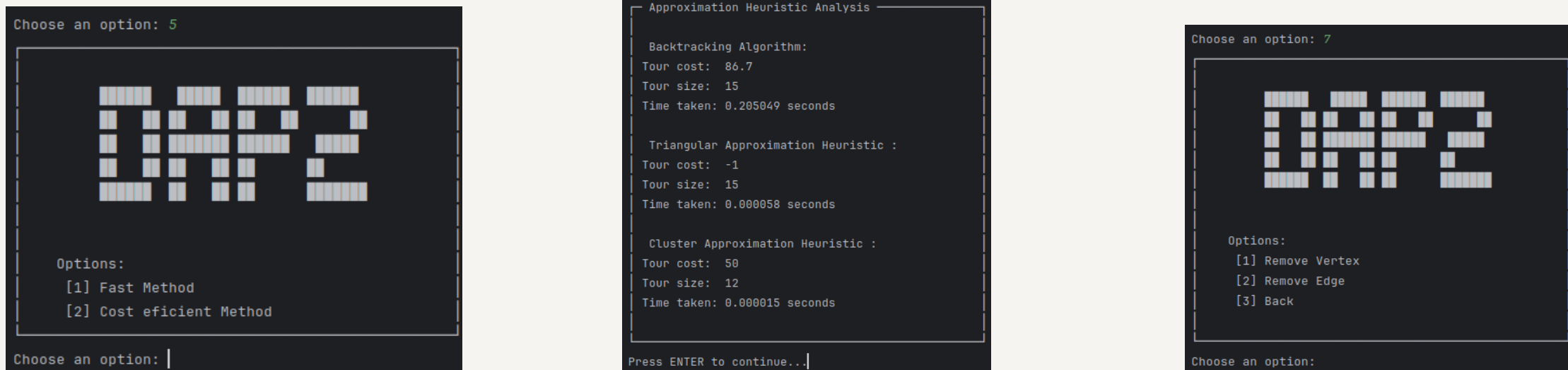
Menu Principal

- Menu 1 – Permite executar o algoritmo de Backtracking, começando pelo vértice com id correspondente a 0.
- Menu 2 - Permite executar o algoritmo de Triangular Approximation, da mesma forma, começando pelo vértice 0.
- Menu 3 - Permite executar o algoritmo de Cluster Approximation.
- Menu 4 - Permite fazer uma comparação rápida entre estes 3 algoritmos comparando o seu tamanho, custo e tempo de execução.
- Menu 5 – Permite correr o algoritmo de TSP nos grafos "Real World".
- Menu 6 – Permite a alteração do dataset, que atualmente popula o programa.
- Menu 7 – Permite a remoção de vértices ou arestas.

Alguns Menus e Exemplos de Outputs



Algumas das opções:



Algoritmos Utilizados

- Triangular Approximation:
 - Este algoritmo utiliza a distância de Haversine para calcular as distâncias geográficas entre vértices. Para além disto, utiliza o algoritmo de Prim para construir a árvore geradora mínima do grafo. Finalmente, faz uma DFS na árvore criada para gerar o percurso inicial. Deste modo utiliza a heurística de aproximação triangular para otimizar o custo do percurso final.
- Cluster Approximation:
 - A heurística de aproximação por cluster para o TSP começa inicializando o tour com o vértice de partida e marcando -o como visitado. Em seguida, itera para encontrar o vizinho não visitado mais próximo, adicionando-o ao tour e atualizando o custo. O tour é fechado retornando ao vértice inicial, adicionando essa distância ao custo total. Finalmente, o estado de visitação dos vértices é reinicializado para permitir futuras execuções do algoritmo.
- Backtracking :
 - Inicializa e prepara todos os vértices do grafo. Posteriormente utiliza a recursão para explorar todas as possíveis rotas a partir do vértice com id correspondente a 0. Finalmente, calcula e compara o custo de cada rota completa, atualizando sempre a melhor rota encontrada. Deste modo, garante que a rota final é sempre de menor custo que visita todos os vértices exatamente uma vez e retorna ao ponto inicial.

Algoritmos Utilizados – TSP Real World

- Tsp Real World
 - Mais eficiente em termos de complexidade:
Bfs para descobrir o vertice mais longe.
Usar Dijkstra para calcular a distância do nó mais longe para todos os outros nós.
Percorre o grafo começando no nó inicial para os vizinhos não visitados com a maior distância até ao nó mais longe.
 - Mais eficiente em termos de custo, porem muito complexo:
Usa o algoritmo anterior, mas usa um algoritmo de Two Opt para tentar otimizar o custo do caminho.

Pseudo Código – TSP Real World

- ```
function tspBranchAndBound(graph, start):
 n = number of vertices in graph
 bestTour = null
 bestCost = infinity

 function branchAndBound(currentNode, currentTour, visited, currentCost):
 if all nodes are visited:
 totalCost = currentCost + cost from currentNode to start
 if totalCost < bestCost:
 bestCost = totalCost
 bestTour = copy of currentTour
 return

 for each neighbor of currentNode:
 if neighbor is not visited:
 visited[neighbor] = true
 currentTour.append(neighbor)
 new Cost = currentCost + cost from currentNode to neighbor

 if new Cost < bestCost:
 branchAndBound(neighbor, currentTour, visited, new Cost)

 visited[neighbor] = false
 currentTour.pop()

 initialTour = [start]
 visited = array of size n initialized to false
 visited[start] = true
 branchAndBound(start, initialTour, visited, 0)

 return bestTour, bestCost
```

- **Função Principal (tspBranchAndBound):**
  1. Inicializa o melhor percurso (bestTour) e o melhor custo (bestCost).
- **Função Recursiva (branchAndBound):**
  1. Completa o percurso retornando ao nó inicial se todos os nós foram visitados e atualiza o melhor percurso e custo se necessário.
  2. Explora todos os nós adjacentes ao nó atual, verifica se são promissores, e continua a exploração recursiva.
  3. Realiza o retrocesso (backtracking) desmarcando o nó como visitado e removendo-o do percurso atual.
- **Inicialização:**
  1. Cria o percurso inicial começando pelo nó inicial.
  2. Marca o nó inicial como visitado.
  3. Inicia o processo de Branch and Bound chamando a função recursiva com o nó inicial.

# Partes Mais Desafiantes

- A parte mais complicada do projeto foi na criação e implementação de algoritmos eficientes que apresentassem um custo baixo e ao mesmo tempo, um baixo tempo de execução para todo o tipo de grafos. Para além disto, a comparação de resultados tornou-se, realmente, difícil devido ao facto da abstinência de resultados modelo para meios de comparação.

# Melhores Features

- Possibilidade de comparar o tempo de execução, custo e tamanho da viagem de 3 algoritmos, sem necessariamente verificar todo o seu percurso.
- Possibilidade de alterar entre "datasets" livremente para uma rápida execução e fluidez de trabalho.
- Possibilidade extra de remover vértices e arestas e testar os algoritmos de backtracking e de aproximação em "datasets" alterados.
- Possibilidade de escolher algoritmos para resolver problemas de TSP no mundo real